Documento Diseño

AngloElectric Project

Integrantes del Equipo: Fiorella Osorio

Cristopher Figueroa

David Valenzuela

Kevin Peña

6.1 Arquitectura

El sistema de Anglo Electric está diseñado bajo una arquitectura de microservicios lógicos orientada a la web, separando claramente las responsabilidades de presentación, lógica de negocio, persistencia de datos y procesamiento de Inteligencia Artificial.

Componentes principales y responsabilidad de cada uno:

- **Capa Frontend (Next.js App UI):** Actúa como el punto de interacción exclusivo para los usuarios finales (clientes y administradores). Construida con Next.js, esta aplicación web se encarga de renderizar la interfaz de usuario de manera optimizada (aprovechando SSR y SSG cuando es necesario), gestionar el estado global (como el carrito de compras y la sesión del usuario) y capturar los parámetros físicos (consumo en kWh y espacio en techo) para la cotización.

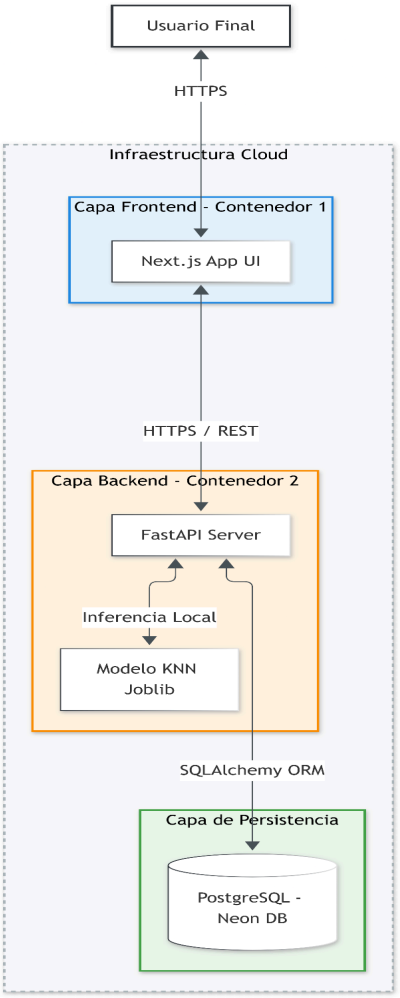
- **Capa Backend (FastAPI Server):** Funciona como el orquestador principal del sistema. Desarrollado en Python utilizando el framework FastAPI, este servidor centraliza la lógica de negocio, procesa la autenticación de usuarios mediante JWT, gestiona las operaciones CRUD del catálogo e inventario, y expone los endpoints RESTful seguros que consume el frontend.

- **Capa de Persistencia (PostgreSQL - Neon DB):** Es el motor de base de datos relacional alojado en la infraestructura cloud serverless de Neon. Su responsabilidad es almacenar de forma estructurada, persistente y segura toda la información crítica del negocio, garantizando la integridad referencial y transaccional de los pedidos, usuarios y productos.

- **Motor de Inteligencia Artificial (Modelo KNN Joblib):** Es el módulo de Machine Learning integrado localmente en la capa backend. Utiliza el algoritmo K-Nearest Neighbors (Scikit-Learn) previamente entrenado y serializado en el archivo binario `modelo_solar.joblib` (apoyado por metadatos en `modelo_solar.json`). Su función exclusiva es recibir variables numéricas de consumo y ubicación e inferir sin latencia de red el kit fotovoltaico más adecuado del catálogo.

Comunicación entre componentes:

El Usuario Final accede a la plataforma a través de internet comunicándose con la Infraestructura Cloud mediante el protocolo seguro **HTTPS**. La comunicación interna entre la Capa Frontend y la Capa Backend se realiza de forma asíncrona mediante peticiones **HTTPS / REST** (GET, POST, PUT, DELETE), intercambiando cargas útiles en formato JSON. Para la persistencia, el Backend (FastAPI) interactúa con la base de datos PostgreSQL en Neon DB utilizando **SQLAlchemy ORM** y el driver **psycopg2 / asyncpg** a través de una URI de conexión cifrada. Finalmente, la comunicación con el motor de IA ocurre a nivel de memoria interna en el backend (Inferencia Local), inyectando directamente las variables al archivo Joblib sin latencia de red.



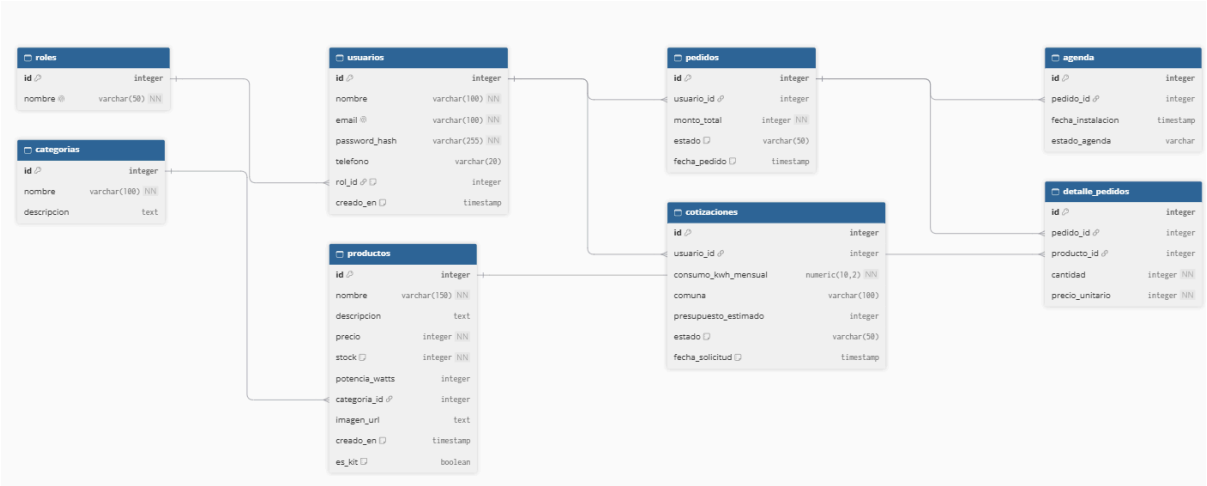
La Figura representa la separación de responsabilidades en la infraestructura cloud, destacando la decisión de diseño de embeber el modelo KNN directamente en el contenedor del backend para realizar inferencia local, reduciendo la latencia, mientras que la base de datos se externaliza a un servicio gestionado (Neon DB) para favorecer la escalabilidad de los datos.

6.2 Modelo de datos

El sistema utiliza un modelo relacional estricto implementado en **PostgreSQL**, ideal para gestionar las transacciones financieras (pedidos) y asegurar la consistencia de los perfiles de usuario y el inventario.

Entidades principales y reglas de integridad del modelo:

- **ROLES (roles):** Define los perfiles de autorización del sistema (Cliente y Administrador) bajo control de acceso basado en roles (RBAC). Restricción de nombre único y obligatorio.
- **USUARIOS (usuarios):** Centraliza las credenciales y datos de contacto de los actores del sistema. Aplica restricción UNIQUE sobre el correo electrónico (email), almacenamiento cifrado irreversible en password_hash y clave foránea obligatoria hacia roles.
- **CATEGORIAS (categorias):** Agrupa lógicamente el catálogo comercial (paneles, inversores, baterías, kits).
- **PRODUCTOS (productos):** Almacena el catálogo técnico y comercial vinculado a categorias. Incluye reglas de negocio críticas como control de inventario y el flag booleano es_kit, el cual filtra los paquetes fotovoltaicos completos elegibles por el motor de IA.
- **COTIZACIONES (cotizaciones):** Registra el historial de simulaciones energéticas de cada usuario (consumo_kwh_mensual, comuna y presupuesto_estimado) procesadas por el modelo KNN.
- **PEDIDOS (pedidos):** Cabecera transaccional que consolida el monto total, estado del pago y fecha de las compras confirmadas por un usuario.
- **DETALLE_PEDIDOS (detalle_pedidos):** Entidad asociativa que desglosa los productos y cantidades de cada compra, congelando el precio_unitario histórico al momento de la transacción.
- **AGENDA (agenda):** Gestiona la programación en terreno (fecha_instalacion y estado_agenda) vinculada directamente a un pedido pagado mediante clave foránea pedido_id.



La Figura muestra el Modelo relacional físico implementado en PostgreSQL. La decisión de diseño fundamental es la normalización hasta la tercera forma normal (3NF), el uso del atributo booleano **`es_kit`** en **`productos`** para optimizar la recomendación directa del modelo KNN sin requerir tablas intermedias adicionales en la cotización, y el aislamiento estructural entre las simulaciones (`cotizaciones`), las compras (`pedidos`) y su ejecución en terreno (`agenda`).

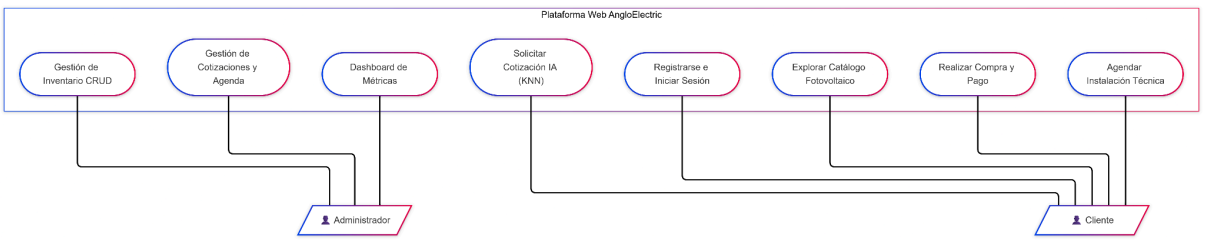
6.3 UML

Diagrama de Casos de Uso

El sistema define dos actores principales con flujos de trabajo aislados:

1. **Cliente:** Sus casos de uso engloban el viaje comercial completo en las rutas públicas y privadas de la tienda: "Registrarse e Iniciar Sesión", "Explorar Catálogo Fotovoltaico", el núcleo predictivo "Solicitar Cotización IA", además de los flujos transaccionales posteriores de "Realizar Compra y Pago (Checkout)" y "Agendar Instalación Técnica".

2. **Administrador:** Operador del negocio cuyas acciones son: "Gestión de Inventario CRUD", "Gestión de Cotizaciones" y visualización del "Dashboard de Métricas".

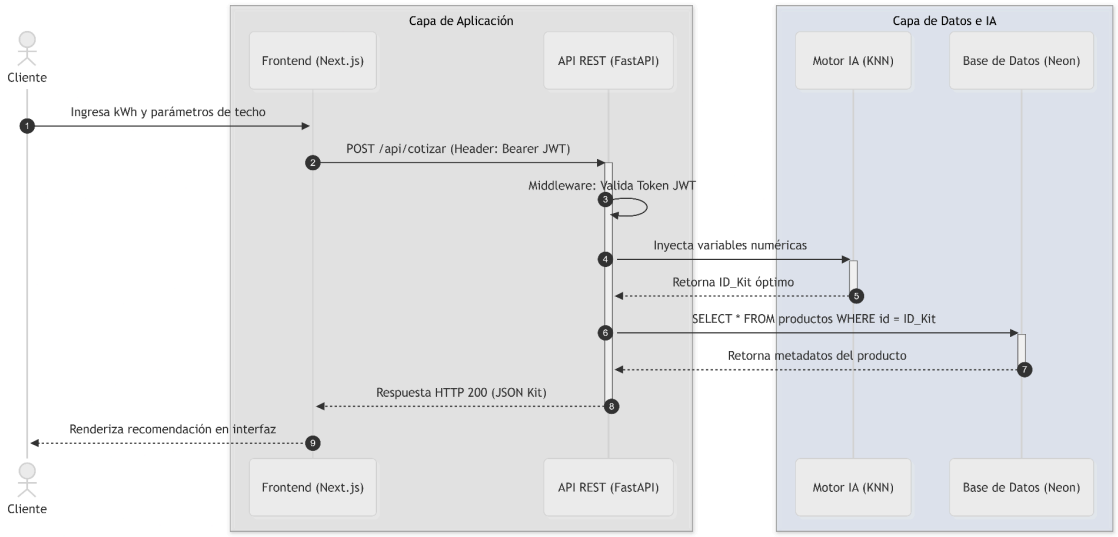


Este diagrama evidencia la decisión de diseño de mantener una plataforma única con fronteras lógicas estrictas basadas en roles (RBAC). El flujo del cliente es netamente de autoservicio y consumo de IA, mientras que el administrador se enfoca exclusivamente en la administración de recursos.

Diagrama de Secuencia de la Funcionalidad Principal (Cotización con IA)

La funcionalidad crítica de la plataforma requiere un alto nivel de orquestación entre subsistemas:

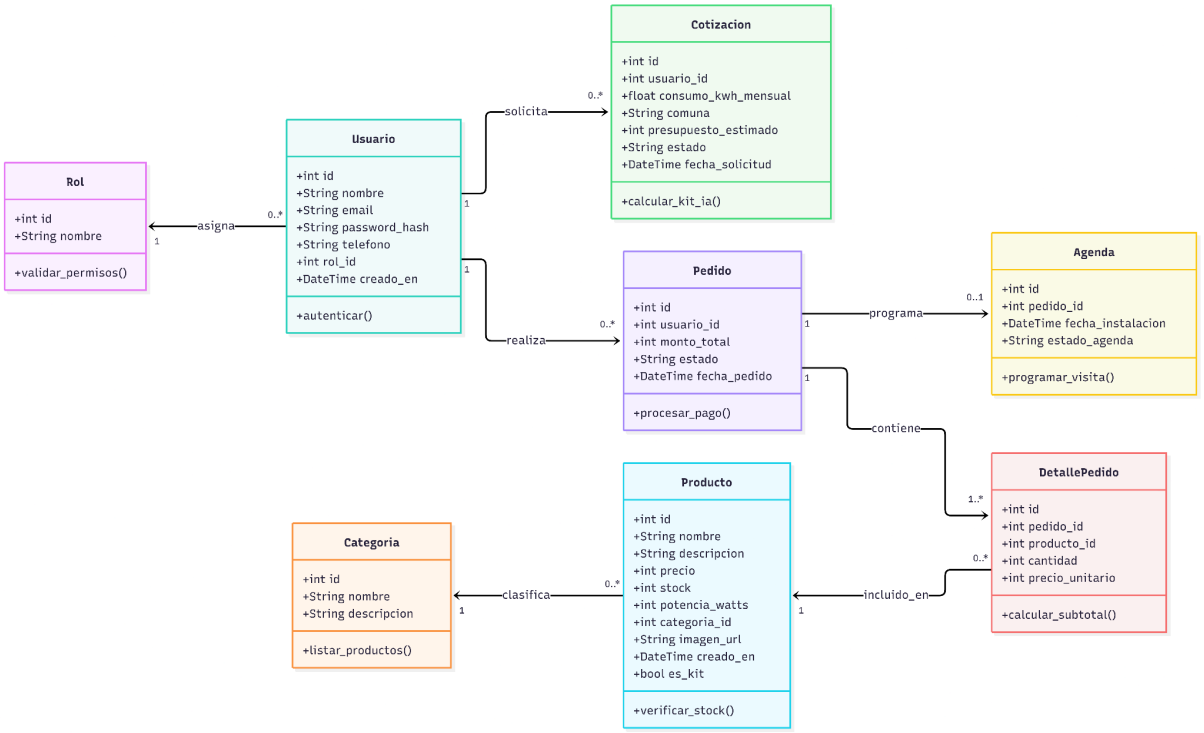
1. El Cliente ingresa sus variables físicas (kWh y parámetros de techo) a través de los formularios dinámicos en el Frontend (Next.js).
2. Next.js empaqueta la solicitud y hace un POST /api/cotizar adjuntando el Token JWT en los headers.
3. FastAPI recibe la petición e invoca su capa de Middleware para validar la autenticidad y vigencia del JWT.
4. Una vez validado, FastAPI inyecta de forma local las variables numéricas al Motor IA (Modelo KNN).
5. El modelo KNN clasifica los datos y retorna al servidor el ID_Kit óptimo.
6. FastAPI toma este identificador y genera una consulta SELECT * FROM productos WHERE id = ID_Kit contra Neon DB.
7. La base de datos retorna los metadatos completos del producto (nombre, componentes, precios).
8. El servidor estructura un HTTP 200 con formato JSON y se lo entrega al Frontend.
9. Finalmente, Next.js renderiza la recomendación técnica en la interfaz del cliente de manera interactiva



El diagrama detalla la decisión de aislar la capa de IA del cliente. Al colocar el modelo KNN detrás del middleware JWT en el backend, protegemos el algoritmo predictivo contra abusos y evitamos exponer la lógica de dimensionamiento al navegador del usuario.

Diagrama de Clases

A nivel de código fuente en el servidor (ORM), las entidades comerciales se modelan como clases interconectadas que heredan de la base declarativa de SQLAlchemy.



Este diagrama de clases ilustra la abstracción orientada a objetos de nuestra capa de persistencia. Muestra cómo se resuelven las cardinalidades (como la relación entre Pedido, DetallePedido y Producto, o la programación en Agenda) a nivel aplicativo antes de ser mapeadas a tablas SQL, garantizando una fácil manipulación de objetos en Python.

6.4 Decisiones técnicas

A continuación se fundamentan las decisiones tecnológicas clave tomadas durante la arquitectura y diseño del sistema:

Decisión	Alternativas consideradas	Razón	Consecuencia
Usar Next.js como Frontend	React.js puro (Vite / CRA), Angular	Permite Server-Side Rendering optimizando la carga inicial, proporciona enrutamiento ágil basado en el sistema de archivos y mejora radicalmente el SEO para la tienda e-commerce.	Requiere un servidor de Node.js activo para alojar la aplicación .
Backend con FastAPI	Django, Flask, Express.js	Excepcional rapidez de ejecución asíncrona gracias a ASGI. Validación automática y estricta de tipos de datos de entrada/salida usando Pydantic. Autogeneración de documentación técnica en Swagger UI, acelerando la integración con el equipo frontend.	Obliga a manejar con especial cuidado las sesiones asíncronas de la base de datos al utilizar SQLAlchemy.
Usar PostgreSQL serverless (Neon DB)	MySQL local, MongoDB, SQLite	Mantiene una integridad relacional estricta . El enfoque serverless permite un despliegue inmediato en la nube con escalabilidad automática según la demanda sin administrar servidores de SO.	Exige una conexión a internet constante incluso para los desarrolladores durante las pruebas en entornos locales.

Seguridad mediante tokens JWT	Sesiones por Cookies (Session IDs)	Permite una arquitectura completamente *stateless*. El backend no necesita saturar su memoria RAM guardando las sesiones de miles de usuarios activos, ideal para ser consumido eficientemente desde APIs REST.	Delega al frontend (Next.js) la responsabilidad de almacenar el token de manera segura y gestionar la lógica de renovación/ expiración.
Inferencia IA con KNN (Scikit-Learn Joblib)	Redes Neuronales Profundas (TensorFlow), Árboles de Decisión	El algoritmo K-Nearest Neighbors es un modelo de clasificación basado en similitud que se ajusta a la perfección para recomendar "kits pre-armados" cruzando perfiles de consumo históricos, con un costo computacional de inferencia sumamente bajo.	Requiere que el ingeniero de datos realice una estricta normalización y preprocesamiento de las variables de entrada antes del entrenamiento, ya que KNN es muy sensible a la escala de los datos.

6.5 Requisitos no funcionales en el diseño

El diseño arquitectónico responde directamente a los atributos de calidad exigidos para aplicaciones modernas de grado empresarial:

- **Seguridad:** Atendido mediante dos vectores. A nivel de datos, las contraseñas nunca se almacenan en texto plano; se utiliza el algoritmo de hash criptográfico irreversible Bcrypt. A nivel de red, absolutamente todos los endpoints de modificación de estado están protegidos por tokens JWT (JSON Web Tokens) de duración limitada.
- **Rendimiento:** Atendido mediante la selección de FastAPI, que corre sobre Uvicorn (ASGI). Esta decisión técnica permite que el servidor procese múltiples peticiones de cálculo concurrente (I/O Bound) de manera asíncrona y no bloqueante, manteniendo latencias bajas incluso durante cotizaciones simultáneas.
- **Escalabilidad:** Atendido implementando el paradigma de microservicios lógicos. Al desacoplar completamente Next.js del servidor Python, y al exteriorizar la capa de persistencia hacia Neon DB (PostgreSQL Cloud Serverless), el sistema puede destinar mayores recursos a los módulos que experimenten picos de tráfico de forma independiente.
- **Portabilidad y Mantenibilidad:** Atendido mediante el uso de la plataforma de orquestación de contenedores Docker. Todo el entorno (dependencias, variables, versiones de sistema operativo) está inmutabilizado, garantizando que el sistema Anglo Electric se ejecutará de forma idéntica en la máquina de desarrollo de cualquier programador y en el clúster de producción de AWS.

6.6 Diseño Docker

Para garantizar el despliegue estandarizado, el proyecto se empaqueta en servicios aislados orquestados a través de un archivo docker-compose.yml.

- **Servicios y su función:**
    - frontend_service: Se encarga de construir y servir la interfaz gráfica de usuario desarrollada en Next.js.
    - backend_service: Aísla y ejecuta el servidor API REST en Python (FastAPI), exponiendo los endpoints comerciales y alojando en su propio directorio los archivos binarios del modelo de Inteligencia Artificial (Joblib).
    - La capa de persistencia no es un contenedor local; el backend se enlaza directamente a través de red a la base de datos gestionada en la nube (Neon DB) para evitar consumo excesivo de RAM en las instancias.
- **Dockerfile de cada servicio:**
    - **Backend:** Se construye a partir de la imagen base oficial python:3.10-slim. Su Dockerfile copia el requirements.txt, instala dependencias críticas como scikit-learn y

uvicorn, expone el puerto estándar interno (generalmente 8000), y define el comando de entrada para levantar la API FastAPI.

    - **Frontend:** Se construye basándose en una imagen ligera **`node:20-alpine`** (compatible con Next.js App Router) para minimizar drásticamente el peso del contenedor. Ejecuta la instalación de dependencias con **`npm install`**, empaqueta la aplicación con **`npm run build`**, y finalmente expone el puerto **`3000`** iniciando el servidor en modo producción.

- **Archivo docker-compose.yml:**
    - Define ambos servicios mencionados. Establece la política de reinicio, realiza el mapeo estricto de puertos de la máquina anfitriona hacia los contenedores , y organiza el orden de inicio de los servicios utilizando las dependencias de red.
- **Redes y Volúmenes:**
    - **Redes:** Se define una red interna tipo *bridge* para asegurar que el frontend pueda resolver al backend a través de nombres de servicio DNS internos de Docker en lugar de requerir IPs públicas.
    - **Volúmenes:** Se declaran volúmenes anónimos condicionales para evitar que la sincronización de las carpetas de trabajo locales (ej. node_modules) sobreescriba los binarios compilados dentro de los contenedores Linux.
- **Variables de entorno requeridas:** El ecosistema depende fuertemente de configuraciones inyectadas. Para su ejecución, Docker Compose solicita un archivo .env con las siguientes llaves (sin valores reales expuestos en el código fuente):
    - DATABASE_URL= (Cadena de conexión completa hacia PostgreSQL/Neon).
    - SECRET_KEY= (Llave criptográfica de alta entropía para la firma de JWT).
    - ALGORITHM= (Ej: HS256 para el esquema de cifrado).
    - ACCESS_TOKEN_EXPIRE_MINUTES= (Entero que dicta la caducidad de las sesiones).
    - NEXT_PUBLIC_API_URL= (URL del backend para ser consumida por el contenedor de interfaz).